

TASK 14

Developing a More Complex Custom System Call in C

Your task is to develop a custom system call in the Linux kernel that performs a more complex operation than simple arithmetic. The system call should accept a string as an input, reverse the string, and then return the reversed string to the user space. This task involves understanding string manipulation in kernel space and how data is transferred between user space and kernel space.

WSL + QEMU + BusyBox – COMPLETE PROCESS

User Space

```
syscall(__NR_reverse_string, input, output, len);
```

Kernel Space

```
SYSCALL_DEFINE3(reverse_string,  
    char __user *, input,  
    char __user *, output,  
    int, len)
```

- **STEP 1: Install Requirements**

```
sudo apt-get update
```

```
sudo apt-get install -y \
```

```
build-essential \
```

```
bc \
```

```
bison \
```

```
flex \
```

```
libssl-dev \
```

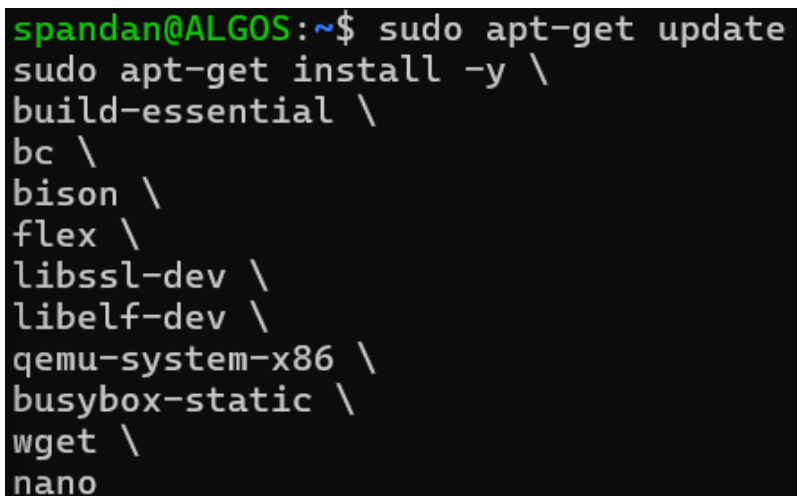
```
libelf-dev \
```

```
qemu-system-x86 \
```

```
busybox-static \
```

```
wget \
```

```
nano
```



```
spandan@ALGOS:~$ sudo apt-get update  
sudo apt-get install -y \  
build-essential \  
bc \  
bison \  
flex \  
libssl-dev \  
libelf-dev \  
qemu-system-x86 \  
busybox-static \  
wget \  
nano
```

- **STEP 2: Create New Working Folder**

```
mkdir ~/syscall_lab_2
```

```
cd ~/syscall_lab_2
```

```
spandan@ALGOS:~$ mkdir ~/syscall_lab_2
spandan@ALGOS:~$ cd ~/syscall_lab_2
```

- **STEP 3: Download Linux Kernel Source**

```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.6.tar.xz
```

```
tar -xf linux-6.6.tar.xz
```

```
cd linux-6.6
```

```
spandan@ALGOS:~/syscall_lab_2$ wget https://cdn.kernel.org/pub/linux/kernel/
v6.x/linux-6.6.tar.xz
--2026-02-03 17:17:45-- https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-
6.6.tar.xz
Resolving cdn.kernel.org (cdn.kernel.org)... 151.101.65.176, 151.101.193.176
, 151.101.129.176, ...
Connecting to cdn.kernel.org (cdn.kernel.org)|151.101.65.176|:443... connect
ed.
HTTP request sent, awaiting response... 200 OK
Length: 140064536 (134M) [application/x-xz]
Saving to: 'linux-6.6.tar.xz'

linux-6.6.tar.xz  100%[=====>] 133.58M  1.21MB/s   in 70s

2026-02-03 17:19:00 (1.91 MB/s) - 'linux-6.6.tar.xz' saved [140064536/140064
536]

spandan@ALGOS:~/syscall_lab_2$ tar -xf linux-6.6.tar.xz
spandan@ALGOS:~/syscall_lab_2$ cd linux-6.6/
```

- **STEP 4: KERNEL CODE**

```
nano kernel/reverse_syscall.c
```

KERNEL CODE

```
#include <linux/kernel.h>
```

```
#include <linux/syscalls.h>
```

```
#include <linux/uaccess.h>
```

```
#include <linux/slab.h>
```

```
SYSCALL_DEFINE3(reverse_string,
```

```
    char __user *, user_input,
```

```
    char __user *, user_output,
```

```

    int, len)
{
    char *kbuf;
    int i;

    if (len <= 0)
        return -EINVAL;

    // Allocate exact length (+1 for safety)
    kbuf = kmalloc(len + 1, GFP_KERNEL);
    if (!kbuf)
        return -ENOMEM;

    // Copy string from user
    if (copy_from_user(kbuf, user_input, len)) {
        kfree(kbuf);
        return -EFAULT;
    }

    kbuf[len] = '\0';

    // Reverse string
    for (i = 0; i < len / 2; i++) {
        char tmp = kbuf[i];
        kbuf[i] = kbuf[len - i - 1];
        kbuf[len - i - 1] = tmp;
    }

    // Copy reversed string back
    if (copy_to_user(user_output, kbuf, len + 1)) {
        kfree(kbuf);
        return -EFAULT;
    }
}

```

```
}
```

```
kfree(kbuf);
```

```
printk(KERN_INFO "reverse_string syscall executed (len-based)\n");
```

```
return 0;
```

```
}
```

```
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ nano kernel/reverse_syscall.c
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ nano kernel/reverse_syscall.c
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ cat kernel/reverse_syscall.c
```

```
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/uaccess.h>
#include <linux/slab.h>

SYSCALL_DEFINE3(reverse_string,
    char __user *, user_input,
    char __user *, user_output,
    int, len)
{
    char *kbuf;
    int i;

    if (len <= 0)
        return -EINVAL;

    // Allocate exact length (+1 for safety)
    kbuf = kmalloc(len + 1, GFP_KERNEL);
    if (!kbuf)
        return -ENOMEM;

    // Copy string from user
    if (copy_from_user(kbuf, user_input, len)) {
        kfree(kbuf);
        return -EFAULT;
    }

    kbuf[len] = '\0';

    // Reverse string
    for (i = 0; i < len / 2; i++) {
        char tmp = kbuf[i];
        kbuf[i] = kbuf[len - i - 1];
        kbuf[len - i - 1] = tmp;
    }

    // Copy reversed string back
    if (copy_to_user(user_output, kbuf, len + 1)) {
        kfree(kbuf);
        return -EFAULT;
    }

    kfree(kbuf);
    printk(KERN_INFO "reverse_string syscall executed (len-based)\n");
    return 0;
}
```

• STEP 5: REGISTER THE SYSTEM CALL

Add to kernel Makefile

`nano kernel/Makefile`

Add at end:

`obj-y += reverse_syscall.o`

```
GNU nano 7.2 kernel/Makefile *
# SPDX-License-Identifier: GPL-2.0
#
# Makefile for the linux kernel.
#

obj-y      = fork.o exec_domain.o panic.o \
             cpu.o exit.o softirq.o resource.o \
             sysctl.o capability.o ptrace.o user.o \
             signal.o sys.o umh.o workqueue.o pid.o task_work.o \
             extable.o params.o \
             kthread.o sys_ni.o nsproxy.o \
             notifier.o ksysfs.o cred.o reboot.o \
             async.o range.o smpboot.o ucount.o regset.o ksyms_common.o

# ++++++
obj-y += reverse_syscall.o
# ++++++
```

Assign syscall number

`nano arch/x86/entry/syscalls/syscall_64.tbl`

Add at bottom:

`548 common reverse_string sys_reverse_string`

```
547      x32      pwritev2      compat_sys_pwritev64v2
# This is the end of the legacy x32 range.  Numbers 548 and above are
# not special and are not to be used for x32-specific syscalls.

548      common      reverse_string      sys_reverse_string
```

Matches user code (`__NR_reverse_string 548`)

Declare syscall prototype

`nano include/linux/syscalls.h`

Add:

```
asmlinkage long sys_reverse_string(
    char __user *user_input,
```

```

char __user *user_output,

int len

);

```

```

//+++++|
asm linkage long sys_reverse_string(
    char __user *user_input,
    char __user *user_output,
    int len
);
//+++++

```

• STEP 6: CONFIGURE & BUILD KERNEL

make defconfig

```

spandan@ALGOS:~/syscall_lab_2/linux-6.6$ make defconfig
HOSTCC  scripts/basic/fixdep
HOSTCC  scripts/kconfig/conf.o
HOSTCC  scripts/kconfig/confdata.o
HOSTCC  scripts/kconfig/expr.o
LEX      scripts/kconfig/lexer.lex.c
YACC     scripts/kconfig/parser.tab.[ch]
HOSTCC  scripts/kconfig/lexer.lex.o
HOSTCC  scripts/kconfig/menu.o
HOSTCC  scripts/kconfig/parser.tab.o
HOSTCC  scripts/kconfig/preprocess.o
HOSTCC  scripts/kconfig/symbol.o
HOSTCC  scripts/kconfig/util.o
HOSTLD  scripts/kconfig/conf
*** Default configuration is based on 'x86_64_defconfig'
#
# configuration written to .config
#

```

make -j\$(nproc)

```

BUILD   arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready (#1)
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ |

```

Verify:

ls arch/x86/boot/bzImage

```

spandan@ALGOS:~/syscall_lab_2/linux-6.6$ ls arch/x86/boot/bzImage
arch/x86/boot/bzImage
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ |

```

• STEP 7: CREATE BUSYBOX INITRAMFS

```
mkdir bbinit
```

```
cd bbinit
```

```
mkdir -p bin proc sys
```

Copy BusyBox:

```
cp /usr/bin/busybox bin/
```

```
chmod +x bin/busybox
```

```
cd bin
```

```
./busybox --install .
```

```
cd ..
```

```
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ mkdir bbinit
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ cd bbinit
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ mkdir -p bin proc sys
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ cp /usr/bin/busybox bin/
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ chmod +x bin/busybox
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ cd bin/
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit/bin$ ./busybox --install .
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit/bin$ cd ..
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ |
```

Create /init:

```
nano init
```

```
#!/bin/sh
```

```
mount -t proc proc /proc
```

```
mount -t sysfs sysfs /sys
```

```
exec /bin/sh
```

```
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ nano init
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ cat init
#!/bin/sh
mount -t proc proc /proc
mount -t sysfs sysfs /sys
exec /bin/sh
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$
```

```
chmod +x init
```

```
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ chmod +x init
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ ll -l init
-rwxr-xr-x 1 spandan spandan 74 Feb  3 17:51 init*
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ |
```

Build initramfs:

```
find . | cpio -o --format=newc > ../bbinit.img
```

```
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ find . | cpio -o --format=newc > ../bbinit.  
img  
4217 blocks  
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ |
```

`cd ..`

• STEP 8: USER PROGRAM

`nano test_reverse.c`

USER CODE

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include <sys/syscall.h>
```

```
#define __NR_reverse_string 548
```

```
int main() {
```

```
    char input[256];
```

```
    char output[256];
```

```
    int len;
```

```
    scanf("%255s", input);
```

```
    len = strlen(input);
```

```
    long ret = syscall(__NR_reverse_string, input, output, len);
```

```
    if (ret < 0) {
```

```
        perror("syscall failed");
```

```
        return 1;
```

```
    }
```

```
    write(1, "Reversed: ", 10);
```

```
    write(1, output, len);
```

```
    write(1, "\n", 1);
```



```
    return 0;
}
```

```
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ nano test_reverse.c
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ cat test_reverse.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/syscall.h>

#define __NR_reverse_string 548

int main() {
    char input[256];
    char output[256];
    int len;

    scanf("%255s", input);
    len = strlen(input);

    long ret = syscall(__NR_reverse_string, input, output, len);

    if (ret < 0) {
        perror("syscall failed");
        return 1;
    }

    write(1, "Reversed: ", 10);
    write(1, output, len);
    write(1, "\n", 1);

    return 0;
}
```

Compile statically:

```
gcc -static test_reverse.c -o test_reverse
```

Copy into initramfs & rebuild:

```
cp test_reverse bbinit/
```

```
cd bbinit
```

```
find . | cpio -o --format=newc > ../bbinit.img
```

```
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ gcc -static test_reverse.c -o test_reverse
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ cp test_reverse bbinit/
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ cd bbinit/
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ find . | cpio -o --format=newc > ../bbinit.
img
6074 blocks
spandan@ALGOS:~/syscall_lab_2/linux-6.6/bbinit$ |
```

```
cd ..
```

- **STEP 9: BOOT KERNEL USING QEMU**

```
qemu-system-x86_64 \  
-kernel arch/x86/boot/bzImage \  
-initrd bbinit.img \  
-append "console=ttyS0 init=/init" \  
-nographic
```

```
spandan@ALGOS:~/syscall_lab_2/linux-6.6$ qemu-system-x86_64 -kernel arch/x  
86/boot/bzImage -initrd bbinit.img -append "console=ttyS0 init=/init"  
-nographic
```

- **STEP 10: RUN THE SYSCALL**

At # prompt:

```
./test_reverse
```

```
~ # ./test_reverse  
I_am_reverse_string_syscall_task_14  
[ 50.861076] reverse_string syscall executed (len-based)  
Reversed: 41_ksat_llacsys_gnirts_esrever_ma_I  
~ #
```

Input

```
I_am_reverse_string_syscall_task_14
```

Output

```
Reversed: 41_ksat_llacsys_gnirts_esrever_ma_I
```

The most recent kernel messages:

```
dmesg | tail
```

```
~ # dmesg | tail  
[ 2.886699] Run /init as init process  
[ 2.887278] with arguments:  
[ 2.887333] /init  
[ 2.887362] with environment:  
[ 2.887388] HOME=/  
[ 2.887421] TERM=linux  
[ 2.993330] mount (48) used greatest stack depth: 14520 bytes left  
[ 3.089342] input: ImExPS/2 Generic Explorer Mouse as /devices/platform/3  
[ 50.861076] reverse_string syscall executed (len-based)  
[ 178.653017] dmesg (51) used greatest stack depth: 14096 bytes left  
~ #
```